

Lec 53 Hands-on on reverse diffusion process

Lec 53 Hands-on on reverse diffusion process

Lecture	53 / 63 + intro
Duration	37:26
Captions	English (auto-generated)
Words	5,438
Video	b1lFJ8_zEdQ
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. We'll be talking about now the second part of the diffusion model tutorial that uh first part we had covered in the last uh tutorial. And let's now move on to the second part of the diffusion model hands-on. We will be talking about primarily the DDPM and the reverse diffusion process moving forward from the forward diffusion that we had demonstrated in the last tutorial where a clean image is um iteratively added with incremental noise and finally becomes um Gaussian noise. And now we will be talking about the reverse process where from a Gaussian noise a image is uh generated using the diffusion model making it a generative AI solution for the generative images. And uh we'll further see that uh how to

[00:51] incorporate the class conditioner. Uh that is in the sense that you would like to have some control on the image that you are generating. What kind of image you want them what class of the images that you want to generate. We will touch upon that and see how to use the class conditional generation using the diffusion model. Um So the notebook covers the reverse diffusion intuition, full DDPM training pipeline, a small unit for MNIST noise prediction loss, uh sampling from the Gaussian noise, classifier guidance and classifier free guidance, uh gradient weight

experiments, uh visual quality comparison, and finally a qualitative discussion on diffusion versus GANs version versus the variational autoencoders or VAEs in

[01:40] terms of the generative application and working principle. Um as usual covered in the previous tutorials, let's let's first make the necessary imports uh in the notebook. Now coming to the diffusion intuition, the forward process uh was there that gradually destroys the structure, right? It starts with the original signal x_0 , and it slowly moves towards the teeth time step x_t . And it gradually keeps on adding noise so that x_t becomes the Gaussian noise.

[02:16] That was the principle in designing the forward diffusion process. Now, the generation starts from approximately the Gaussian noise, that is the reverse process, and it repeatedly estimates the less noisy state. So, at every step, it predicts the noise that needs to be subtracted out. So, to get the less noisy image. And you keep on doing that, and you should get back x_0 starting from x_t . The [snorts] network predicts the noise present in the current image u_t which is parameterized as $\epsilon_{\theta}(x_t, t, y)$. So, given the class level y , the time step t , and the current noisy image x_t , the ϵ_{θ} is the predicted noise. θ is the network that you build to predict this noise ϵ .

[03:03] Um hope the notations and u_t settings are clear. Now, let's move to the DDPM equation that might have been covered in the theory course. So, uh so, it uh it has a mean of $\mu_{\theta}(x_t, t)$ defined by this equation. So, u_t you can look at the equation and see that the mean μ_{θ} is parameterized by θ , which is the network that you learn to predict the noise. And it uses the parameters α_t and β_t uh and the current image x_t .

[03:37] Um and also it basically uh depends on the mean $\epsilon_{\theta}(x_t, t)$ that you are predicting. So, the quality of this mean estimation is exactly dependent on the a network ϵ_{θ} that is predicted by the network. A reverse sample is expressed as $x_{t-1} = \mu_{\theta}(x_t, t) + \sqrt{\beta_t} z$ where z is a standard normal uh with β_t defined using this expression. So, at the final step no additional random noise is added and you finally get the clean image.

[04:23] Okay. Now come uh let's come to the diffusion schedule. A short linear schedule we will use here. Production models might use you know complicated uh and faster methods. So, let's define uh t to be 100 which is the number of time steps. Uh we'll have we have defined something called fast mode in the in the at the start of the tutorial. Here, right? So, if you set it to false, it will increase the number of time steps. We have kept it to uh true and made sure that the notebook runs pretty quickly u_t using this fast mode setting.

[05:00] Um okay. Now let's clear uh create the linear linear variance schedule. So, the β_t s are nothing but the linear u_t linear separation linear uh linear steps u_t

leading to 100 uh which is t and then alphas are one minus betas uh simply and um you know all the all the expressions with respect to α bar and μ and you simply calculate the posterior variance and you know betas range. So, you can see that the diffusion steps is 100 and the beta uh ranges from zero to 0.02.

[05:37] Okay. Um Now we can also plot uh the alpha and beta variations. You can see alpha starts from one and then keeps reducing whereas beta starts with zero and keeps increasing with the time steps. Now you define two functions called `extract` and `Q sample`. The job of the `extract` function is simply to convert those coefficients in into the shape of reshape them to the image dimensions so that you can simply multiply it allowing matrix operations.

[06:11] Um and here is a `Q sample` function which samples a noisy image during noisy image directly from the posterior of Q of X_T given X_0 . This function was again implemented in the previous tutorial as well. And So you you define the posterior here. Okay, now let's let's load the MNIST. We'll we'll primarily use MNIST for our purpose of this tutorial which makes it short, but yeah, for original images, you are welcome to scale this tutorial to real real-looking images like other data sets. So MNIST is small, so let's go ahead with that.

[06:51] We finally similar to previous tutorials, we load the data set and define the train size to be 12,000 and validation side validation size to be 2,000. And you define the train and validation data sets using PyTorch's data set class and you finally create the data loader instance. Train loader and validation loader at the data loader corresponding to the train split and validation split. You finally as a check print the training and validation sizes which finally gives you 12,000 and 200 2,000 as you intended it to be.

[07:31] Um for your own visualization, let's let's you know display couple of data points from the MNIST data set that you have already loaded. You can see that different digits from the MNIST samples which yeah, you have looked at I'm sure uh many times before. Now comes the unit part. So so unit is a neural network which consists of a encoder followed by a decoder. Uh it essentially is a set of convolution layers which first keeps on reducing the dimension of the image and then finally um comes to a bottleneck layer and from the bottleneck layer it keeps on uh keeps on unpooling it and increasing the dimension. So it's essentially the dimension keeps reducing and then starts increasing leading to a

[08:23] encoder-decoder unit architecture. Also the skip connections are used uh like a resnet to preserve the local information which are essential. You might have uh studied about it in vision courses. The model also receives couple of more arguments as we have seen which is which are required to make the prediction of the noise. So first is the time step embedding that represents the current noise level and the class embedding that specifies the desired digit. So this is class conditional Y we had talked

about.

[09:00] Okay. Uh now now our first job is to create the time step embeddings. Time step embeddings like transformers you have read. It uses in the positional embedding people use the sinusoidal function to create the time step embeddings. Uh so so similarly sinusoidal time embedding function is created which simply takes the sinusoidal function of the uh sinusoidal function on the time step uh T that are that are there in the diffusion steps.

[09:34] And uh finally we need to define a residual convolutional block um, conditioned on time and class features. So, so that neural network, um, first use, uh, you know, if you look at the forward function here, um, it uses, uh, the first the residual projection and then, uh, set of convolutional layers, conv1 and conv2, and finally returns the hidden plus residual. And, uh, and you can see this hidden equal to hidden plus self.condition_projection_of_condition.

[10:08] This incorporates the conditional information that we have talked about here, that you need to provide the time step and the class embedding as the arguments in the network to predict the noise correctly. And, uh, this is the step which does that. And, um, so this line that hidden equal to hidden plus self.condition_projection, um, condition is the set of, um, information that we have talked about, the arguments that need to be passed to predict the noise correctly, that at the time step and class embedding informations.

[10:43] And, uh, let's I have also, uh, you know, incorporated a pseudo code level, uh, implementation or visualization of this function. So, image features come in and then you have a a set of norm uh, group norm function and silu activation with, um, with a set of convolution layers, 3 cross 3 convolutions, and then you need to add the time step and class condition to predict the noise as we have seen through the closed form expressions of the noise. And, uh, after that you need to, uh, we we we use the group norm again and then, uh, non-linear activation as usual and then set of convolution once more. And, uh, there is also a skip connection which which, uh, connects the input features directly to the final prediction layer,

[11:29] uh, making sure that the local features are preserved. Yeah? So, that leads to the output features from the unit. Now, we need to we need to make a neural network which will simply predict the noise directly. Okay? On top of the previous previous network that you have created. So, let's define small conditional unit which defines a compact class conditional unit for the MNIST. So, so you can look at the structure of the network. It it consists first defines the number of classes and here is self.non null class which is used to do unconditional generation. What do you mean by unconditional generation is that you don't want to provide any class levels. You simply want to do free form generation in case of MNIST. You For

[12:21] example, if you don't want to generate a particular digit. You all you want

is generating random digits from MNIST, but they should look like MNIST. But, you don't want to class condition it on particular digit like five. Okay? So, that time you need to set the null class. If you if you generate using null class, then it will do the free form generation. Similarly, you define the time embedding using the sinusoidal time embedding function you had created.

[12:47] And uh and then a set of projection and a projection and class embedding and then some set of encoder and decoder blocks with the down sampling and up sampling functions. Yeah? And you also need to define the bottleneck layer dimension of the unit that we have already talked about. Coming to the forward function, first the condition that at the time step and the class levels that are defined in condition. Yeah? » [snorts] » And then the input features is the input projection of the noise image that you have. And uh you can define the skip one hidden and skip two and set of hidden um uh consecutive layers after using the upsampling uh upsampling and decoder blocks. Uh so, first encoders go in and then decoders

[13:43] come in. Uh so, as I said, it's a encoder-decoder architecture in case of U-Net. And then finally, you get get back uh get yourself that output layer of hidden, which is supposed to be the predicted noise. So, a diffusion model is small conditional unit, um and it is set sent to the GPU using two device, and um you can print the model size. It turns out to be about um 778k uh order parameters. So, now we need to we we here uh show uh us the dimensions that are getting used.

[14:19] So, you you can call test images and test levels from next iteration of train loader in sense. And test images and test test levels also you can sample from the data set and loader. And um if you predict the uh shapes of the images, so noisy image, um um predicted noise, which has to be the dimension of the image, uh and the target noise, all are of same shape. So, you can you have make sure through this quick check that um the U-Net architecture is predicting the noise of same shape uh as the image.

[15:03] Okay. Now, coming to the DDPM training objective. Uh DDPM training objective, as we had we touched upon this last time uh towards the end of the previous um tutorial on um diffusion. So, the DDPM objective is L_{simple} , which is the loss that needs to be minimized is the expectation of $\epsilon - \epsilon_{\theta}(x_t, t, y)$. Uh, that difference um squared error of that uh of that error. Squared error between the actual noise ϵ which were the noise that were added to the clean image uh step by step to come to the Gaussian noise in the forward process, that stepwise noise you want to predict using ϵ_{θ} where θ is the unit parameters in this case.

[15:52] Training steps involve sample a clean image and label, then sample a random time step, sample a Gaussian noise and construct the noisy image, and occasionally replace the class with a null condition uh just to make it uh just to make sure that it it can also generate MNIST uh images unconditionally. And predict the noise finally,

and um based on the prediction, whatever is the mean squared error to the with respect to the actual uh noise that was added in the forward function, just minimize the squared error loss and use this as a loss to train your unit architecture.

[16:33] As I said, the null condition is required for classifier free guidance. We now prepare the training batch which takes the arguments of clean images, class levels, and drop probability. Um dropout probability as um as point one, and then uh you can define uh the batches of the data set. You can go through this implementation. I will not cover it line by line for uh time's sake. And uh you can also define a valid evaluate function on the diffusion loss.

[17:12] Um you you you can define the evaluation function, and then after the evaluation mode is done, you can you need to set the model back to the training mode. Whereas at the start of the function, you always need to set the model to eval mode to make sure that the gradients are not stored in the in any evaluation function of any neural network. So, in the evaluation function, you can see that the clean images are loaded to the GPU and then the class levels are all loaded to the GPU as well and the time steps as well. And then the noisy image and true noise, you can sample from the Q sample function that we had created.

[17:55] And coming to the predicted noise, that predicted noise noise simply is obtained by passing the noisy image, time steps, and class levels through through the model of unit. And you you can compute if {dot} MSE loss of the predicted noise and the true noise and then you add the loss to it and then whenever we are training the model, we will do the loss {dot} backward, which you will see in the next part. You need to also define a optimizer function to optimize the model using gradient descent.

[18:31] We we use Adam optimizer with a starting learning rate of to the power minus two 10 to the power minus three. Um we set the number of epochs to be four to make the keep the tutorial short and fast running. And we set the model diffusion model {dot} train. And then for every epoch, for every batch obtained from the train loader, you can see that the loss is computed using if {dot} MSE loss between predicted noise and true noise. And you need to set optimizer {dot} zero grad and you do loss {dot} backward and optimizer {dot} step. These ensure that the gradients are calculated with respect to the squared error loss between predicted and the true loss and that that gradient is used to update the

[19:19] model parameters. And then you after every epoch you need to calculate the validation loss to make sure that the model is not over fitting as usual for training of any neural network. Um you need to uh you need to track both training and validation losses to make sure the training is happening properly. You can observe that the MSE loss keeps reducing in training cases. Uh it reduces from 1.09 to 0.07 and also you can see that the validation MSE also keeps reducing. It starts with 0.01 and goes to 0.07.

[20:05] Now uh we plot the training behavior. You can see that uh MSE loss with respect to training and validation uh keeps reducing. Here we plot the training MSE loss. You can save the model using such a utility function. I have provided the code for it in case you want to save the model and use it later. And here we go for the reverse sampling from the Gaussian noise. Um as I said the starting uh of reverse process starts with a Gaussian noise x_T which is sampled from the standard normal Gaussian noise. The model is called at every time step and the reverse equation produces the next less noisy image. And in the reverse process you keep it repeat keep on keep on repeating the step till you uh till you achieve the

[21:01] uh clean image completely. Again you need to define the P sample function. And There are two things that are possible. One is unconditional noise where simply the model is provided with the noisy images, time steps, and null levels. Whereas the conditional noise um conditional noise is nothing but um noisy images, time steps, but the class levels you need to provide. In case of unconditional noise, as I said, we we kept a set of class called null levels, which essentially does not guide the model towards a particular digit or particular class. So, you just provide the null levels to get the unconditional noise. Whereas, if you want to make sure that the noise that is getting generated will keep denoising

[21:54] the Gaussian noise towards a particular digit, let us say five, then you need to provide five in the class levels. And you can define the guided noise in that case to be unconditional noise plus guidance scale multiplied by conditional noise minus unconditional noise. So, as you can see, this is a simple linear scale of providing more and more guidance where the scale or extent of guidance that intended to be provided has to be provided as a guidance scale. If you keep increasing the guidance scale, the guidance is more toward the particular class. But having too much guidance scale can harm the quality of the image.

[22:33] That we So, there exist a trade-off. Then you can usually get all your β_T , α_T expressions using the closed form expressions that are provided repeatedly in the previous cells. And you finally provide the denoised image. So, let us take seven as a trajectory reverse trajectory example. The The level is seven. So, trajectory noise is uh sampled from the torch.random and generated digit and trajectory states can be obtained by calling sample images function. Uh where you provide the diffusion model as the model trajectory level um that is a class level that is given. Guidance scale we are taking as two.

[23:23] Uh the initial noise is trajectory noise and uh and we can see uh the uh with time steps starting from t equal to 99. So, this is a reverse process, okay? So, you start with t equal to 100 and go to t equal to zero. So, you can see that um initially it was a complete random noise at the left. As you keep progressing, you can see that the towards the end uh again, this is a very primary level tutorial, so um it doesn't look like really seven, but what you essentially can pick from this quick demo is that

slowly the noise start to look like a digit, at least a MNIST digit-like structure.

[24:03] So, given more iterations, stronger network, and uh more training, this will look like a digit, but essentially the flavor you can pick is that from a completely um random Gaussian noise, how how this reverse diffusion process can generate such a digit-like MNIST digit-like structure given this simple straining. Now, there are two kinds of guidance that are provided. One is classifier guided, another is classifier free guidance.

[24:33] Um we have covered the classifier free guidance. Um but here it is. So, classifier guided diffusion is another possibility where a separate noisy image classifier supplies a gradient proportional to the um gradient of $\log p(y|x)$. So, with respect to the class levels that are possible, uh the gradient is calculated for a particular noisy image x_t for the class conditional and that is used as the guidance. So, whereas as we have covered and showed the implementation to you, the what what we have implemented is classifier free guidance. So, there is no classifier, but you essentially have the same diffusion model is trained with and without label and the predictions are combined as

[25:21] ϵ_{hat} equal to $\epsilon_{\text{unconditional}}$ plus the guidance scale ω multiplied by $\epsilon_{\text{conditioned}}$ minus $\epsilon_{\text{unconditioned}}$. That's what we have implemented. Uh, here we make a comparison with the gradient weights. Uh, it's a simple utility function just to compare the guidance scales. Um, you can go through this and we assume so every row is with respect to one guidance scale. First row starts with guidance scale equal to zero, which means unguided completely and and there are other three rows which correspond to guidance scale of one, two, and four.

[26:10] What you can observe, uh, let us let us look at the first column, right? So, for the class zero, you can see that um, for guidance scale equal to zero, uh, the image don't really look like zero. But when you go to the last row, it really looks like a zero, right? So, so it So, the class conditional makes it that particular digit like structure. So, uh, but again, it's not clearly visible in case of some of the other classes.

[26:46] For example, um, so here you can see at least for two again, it's it's nowhere close to looking like a two, but at least it is trying to intend to go make a assume a structure like two slowly when the guidance scale is increased. I think Six here is another where it doesn't at all look like a six, but it's taking slowly a structure of six in case of the last row. And with the again as I said with the more more data and larger scale training, it will it will be more visible which you can take up as an exercise.

[27:35] And here is something that we go towards the evaluation side of the diffusion models generated images. So the diffusion model generates a lot of images and then you want to maybe like check how good the digits are. The first first thing that we are doing also here is that going through the digits you know visually and then making a qualitative assessment, but for a large scale generation, you need to have a

quantitative assessment. How that is done is that you take the generated digits and then train a network and then see you essentially create two splits, train split and test split and then if you train on the train split a network which will simply recognize the digits and classify the digits on the test set if that same

[28:25] model is able to validate and do the classification properly. So you just create a small MNIST classifier using the very standard CNN style classifier which we call as quality classifier. Image quality classifier and then you train the classifier. Yeah, so on the training you achieve a very high accuracy, but when you go to evaluate the generated images on the test split, you can see that the accuracy keeps on increasing. It's a very small scale, only 10 images are taken, so you get although you get a 0.3 accuracy on guidance scale of zero.

[29:10] Um It might be that that it's noisy. So, given larger images, the trend will be visible at least in this small scale experiment. This guidance scale of one, two, and three at least shows the trend that with increasing guidance scale, the generated digits become more recognizable because they're not free scale generation anymore. Free form free guidance generation anymore. And coming to loss function choices, we use the We use the mean squared error, right?

[29:49] But other systems there are other models and training schemes that are proposed by other works which use a clean image which might predict a clean image rather than the error. And it might use a might use a velocity parameterization and weight combinations based on signal to noise ratio that is happening with the steps of the diffusion model. I leave it to you for further reading on this and in case you want to explore more on them, so thought of mentioning them.

[30:25] And so here we are close to the last part of that this hands-on session where we'll we'll also point you towards now the diffusion GANs and VAEs, variational autoencoders, what are the differences in the operating principle of them. Now that you have learned the diffusion model, how it is it different from the previous works and more popular models on generative AI, which are GANs and VAEs. So, coming to the training signal, diffusion uses a denoising regression, whereas GAN uses adversarial game where uh there is a there are two networks, um one discriminator and the one generator where each uh each of them is trying to fool the other. And for VAE, there is reconstruction plus latent regularization.

[31:16] Coming to its training stability, diffusion is comparatively much more stable because it uses a convex function of MSE error, whereas it's widely known that GAN's training is very unstable because of this conflicting training uh regime. So, it's it's very unstable in that sense. Uh and for VAE, it's usually stable. Coming to the sampling speed, uh diffusion is again iterative. That's why it's slower in generating the images, whereas GAN is usually one pass, so it's fast. And VAE is also one decoder pass, which is so GAN and VAE are much more efficient in terms of time, although

they their training is difficult and um their uh their quality of the generated images might be compromised.

[32:07] And coming to model uh coverage, a model coverage is often strong in case of diffusion. Um whereas GANs might uh suffer from mode collapse, uh which is also again one of the biggest problem of GANs. And uh VAE also is quite widely uh can uh has a good uh um mode coverage. And coming to uh image sharpness, diffusion has really good quality sharp images. Um GAN also has that, but VAE often tend to generate smoother and blurred blurred uh images.

[32:44] And the main practical cost, uh so con- considering the compute perspective, diffusion has many denoising calls calls making it computationally a little bit expensive. GAN is adversarial optimization, which is which requires very rigorous training, and VAE it it has uh reconstruction and quality trade-off. So, these these are the broad tendencies rather than the universal rules, and depends on the particular implementation and training schemes being proposed.

[33:15] So, I have also noted down couple of exercises for you, like varying the guidance values, and then remove one um unit skip connection, and then retrain and see the effect of using the skip connections. And uh try condition drop probabilities by varying it uh and increase the T from 100 to 200 to see how how the increasing the number of uh diffusion process really increases the quality of the generated image. And then you can you will also try the number of base channels. You can uh vary from 16 32 64. And uh reuse the some initial same initial noise while changing only guidance. And you can uh essentially do the disable the fast mode so that it goes towards this uh more epochs, more number of steps, and so on.

[34:07] And um and I I would also encourage you to try a SNR weighted version that we have already talked about as the other variants of uh losses that are used to train the diffusion process reverse diffusion process, and compare it with the compare the quality with the MSE loss that we are using in this notebook as the most widely used um policy. Um Coming to common failure modes, uh, sample uh uh, remain noisy. The model may be under trained too small or using an incorrect reverse equation. So, in that case, samples will remain noisy.

[34:42] Every level produces similar level images. So, in that case, the network may be ignoring the class embedding. So, the class conditional is not uh, effectively getting done and the guidance has little effect. Uh, in that case, conditional and unconditional predictions may not yet differ meaningfully. And uh, large guidance creates artifacts. If you provide a lot of uh, class uh, guidance weight, then the uh, the basic naturalness of the getting uh, of the generated images might be compromised.

[35:17] Uh, yeah. So, and and one finally is loss decreases, but samples remain poor. Noise prediction uh, mean squared error is a local objective. So, good sampling also requires adequate capacity time stamp coverage and a correct reverse process. So,

loss degradation is merely merely a uh, quantification or visualization with respect to the training is happening with respect to the loss or not, but it the final quality of the samples generated sample depend on these other factors that are measured.

[35:52] Um, coming to the uh, quick summary, revis- reverse diffusion process repeatedly transforms the noise towards the There is a unit architecture which combines the global structure with the fine spatial details. The DDPM training commonly predicts known Gaussian noise using the mean squared error. The time step condition defines the correct current noise level. Classifier free guidance combines the conditional and unconditional predictions. Moderate guidance may improve class adherence at the same time keeping the preserving the quality.

[36:27] Excessive guidance may reduce diversity or create artifacts. A classroom uh yeah. So, the model that we have created is it simply demonstrates the mechanism, but it's not a yeah, production level uh model, and I really urge you to uh increase the scale at your own end and then study uh the effect of these parameters and then uh take up a larger project on uh diffusion model to see the real-time beautiful images the diffusion model has the capacity to generate and uh and make your own implementation and get your hands dirty.

[37:07] Uh that's all for diffusion models. I hope this part two uh completed the hands-on on diffusion that we started as a part one, which was the previous tutorial. And hope it was helpful. Thank you.